CIC-307 / COMPUTER NETWORKS MINI-PROJECT

NetArchitect

Recommendation & Config Engine

Project Handbook and Technical Briefing

An explainable planning utility that turns network requirements into justified media and routing choices, a bitwise VLSM address plan, and reviewable Cisco IOS drafts.

CAMPUS LAN	METRO MAN	GLOBAL WAN
1,600 endpoints	450 users / dual-homed	4 cities + cloud

Prepared for the CIC-307 project submission
Handbook date: 31 October 2026 | Assignment deadline: 10 November 2026

SOFTWARE VERIFICATION RECORD: 26 UNIT TESTS PASSED; PRESET AND CUSTOM EXPORT CHECKS PASSED

How to use this handbook

A presentation-ready explanation of what was built, how it makes decisions, what was checked, and where engineering review is still required.

SECTION	TOPIC
01	Executive summary and project fit
02	Architecture and functionality inventory
03	Three assignment scenarios
04	Decision logic: media, VLSM, routing, and ACLs
05	Outputs, commands, and operating workflow
06	Feasibility, uses, and practical value
07	Verification evidence, risks, and limits
08	Presentation notes, likely questions, and final summary

Executive summary

NetArchitect is a local Python command-line application for early-stage network planning. A user selects the Campus LAN, Metro MAN, or Global WAN profile, or supplies a custom JSON profile. The engine validates the inputs, selects a media and routing design, allocates IPv4 subnets using VLSM, and writes a decision report and Cisco IOS configuration drafts.

The project is designed to make each recommendation explainable. A report states the rule, the chosen value, the calculation assumptions, and what must be checked before deployment. The same computed plan feeds the report, CSV, JSON, switch configuration, and router configuration, which keeps addresses and policy references synchronized.

> Project position: a feasible, low-cost educational planning prototype with a passing 26-test unit suite and successful preset/custom export checks. It is not a live network discovery, simulation, or deployment system.

The assignment is a 100-point CIC-307 project, due 10 November 2026, and may be submitted individually or by a team of two. It explicitly permits AI-assisted scaffolding while requiring students to validate algorithmic boundaries and defend design choices. The audit section documents two flawed draft ideas, their corrections, and packet-flow reasoning. The attached assignment brief is treated as a project specification, not as executable code or an authority to deploy configurations.

Assessment rubric and evidence

Assessment area	Points	Project evidence
Parameter engine and presets	15	Three profiles; custom distance, density, resiliency, and address validation.
Explainable decision engine	25	Media type and optic rationale, routing protocol, topology tier, and latency/SLA caveats.
Subnet engine and configurations	25	Bitwise VLSM, non-overlap, gateway/wildcard math, IOS switch/router/SVI/ACL drafts.
Automated unit testing	15	26 passing tests across distance, exhaustion, wildcard, routing, validation, and output sync.
AI code audit and defense	20	Two corrected draft failures, packet-flow walkthroughs, post-mortem, and viva notes.
Total	100	Assignment score allocation.

1. Project purpose and assignment fit

The brief asks for a scenario-driven tool that explains decisions rather than emitting unexplained configurations.

The product goal is to reduce repetitive first-pass planning work: translate distances and endpoint counts into a structured design, expose address math, and create draft device configuration that a student can review and defend.

Assignment need	Project response	Evidence in repository
Three preset scenarios	Campus, Metro, and Global inputs with scale-specific network focus.	netarchitect/profiles/*.json; examples/profile folders.
Custom parameter engine	Validates IPv4 pool, endpoint count, switch density, resiliency, link distance, media, VLANs, and subnets.	models.py; validation.py; generate-custom
Explainable media choices	Cat6a / 10GBASE-T through 100 m, OM4 / 10GBASE-SR to 400 m, OS2 beyond; includes optical assumptions.	media.py; report.md
Boundary failure handling	Copper beyond 100 m raises PhysicalDistanceViolationError with signal-loss context.	errors.py; test_media.py
Explainable subnetting and routing	Descending VLSM with gateway reservation; OSPF/eBGP choice with rationale and wildcard masks.	vlsm.py; routing.py; ALGORITHMS.md
IOS configuration generation	Layer 3 switch and edge-router drafts with VLANs, SVIs, ACLs, OSPF, and optional QoS/BGP/IPsec.	configgen.py; examples/*/*.cfg
Unit testing and AI audit	26 passing unit tests plus two reviewed AI failure cases and ACL flow walkthroughs.	tests/; docs/ai_audit_log.md

Scope choice

The assignment allows a CLI, web dashboard, or API. The project uses a CLI because it runs locally, needs no server or third-party runtime library, and keeps the algorithmic steps inspectable. The planning engine is separated into modules so a web interface or API could be added later without rewriting the calculations.

Non-goals

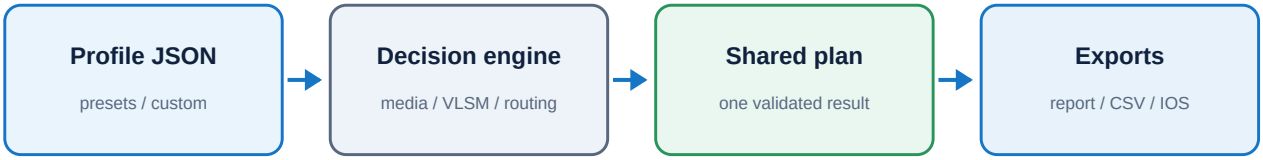
- No live topology discovery, router login, configuration push, or credential storage.

- No packet capture, full network simulator, or guarantee that a Cisco command is supported on every platform.

- No carrier guarantee for the Metro 5 ms objective and no final fiber-budget guarantee without real optics and path data.

2. Architecture and functionality

A small pipeline keeps the decision logic modular while rendering every artifact from one shared plan.



All artifacts are rendered from the same allocation and decision record.

Processing sequence

- Load a built-in profile JSON or parse a custom JSON file into immutable profile, link, subnet, and ACL records.

- Validate inputs before producing recommendations: CIDR alignment, IPv4 scope, endpoint counts, unique VLANs, media values, routing areas, and BGP/VPN references.

- Run the media engine, VLSM allocator, and routing selector; collect their results and assumptions in one plan object.

- Render the Markdown report, CSV address table, JSON plan, core-switch draft, and edge-router draft from the same plan.

- Optionally walk one source-SVI ACL decision with the packet-flow evaluator.

Module	Responsibility	Why it matters
cli.py / __main__.py	Preset selection, custom profile command, exports, and direct-file or package entry point.	One simple interface for classroom use and demonstrations.
models.py / validation.py	Typed inputs and early error checks for profile, VLAN, link, routing, BGP, and VPN fields.	Rejects unsafe or ambiguous input before writing config.
media.py	Distance-based medium/optic recommendation and generic link-loss estimate.	Makes media boundaries and assumptions visible.
vlsm.py	Largest-first aligned IPv4 allocation with gateway, broadcast, capacity, and wildcard data.	Prevents overlap and undersized subnet blocks.
routing.py	Single-area OSPF, multi-area OSPF, and eBGP boundary recommendations.	Connects topology scale to a stated routing rationale.
configgen.py	Renders switch/router IOS drafts from assigned addresses and rules.	Keeps configuration addresses consistent with the report.
engine.py / packetflow.py	Builds the shared plan, writes outputs, and evaluates an example ACL flow.	Provides repeatable exports and a transparent policy walkthrough.

Single-source rule: allocation happens once. Reports, CSV, JSON, SVI addresses, OSPF statements, and ACL wildcards all consume those same allocation records.

3. The three planning scenarios

The presets turn the assignment profiles into concrete but reviewable address and configuration examples.

Profile	Scale and topology	Design choices	Address/routing focus
Campus LAN	3 buildings; 1,600 endpoints; short access and campus-backbone links.	Cat6a access, OM4 multimode backbone, VLAN segmentation, rapid-PVST, PoE-aware notes.	10.10.0.0/20; one OSPF Area 0; Guest-to-Management ACL example.
Metro MAN	HQ, two remote offices, and a FinTech colo; 450 users; four logical sites.	OS2 at 35 km, EVPL at 15-32 km, 5 ms target, WAN QoS example, lower OSPF cost on primary path and higher cost on diverse backup.	10.20.0.0/20; multi-area OSPF with site area IDs and routed handoffs.
Global WAN	Four cities and one cloud boundary; 2,000 IoT/scanner endpoints plus supporting segments.	Carrier WAN, OS2 data-center handoff, IPsec template, multi-area OSPF, eBGP, reserved summary.	10.30.0.0/16; AS 65010 to sample peer AS 65020; remote VPN prefix 10.31.0.0/16.

Campus LAN

The campus profile represents 1,600 planned endpoints across staff, students, voice, access points, IoT, guest, and management segments. The address plan reserves the first usable address as each SVI gateway. The generated switch draft includes 802.1Q trunking, rapid spanning tree, VLANs, and single-area OSPF. Short access runs use Cat6a; building backbones at 260 m and 360 m use OM4 and 10GBASE-SR.

Metro MAN

The Metro profile represents 450 users across HQ and two remote offices, with a FinTech colocation handoff. The design separates HQ, East, West, and colo areas. The West site has a primary EVPL path and a diverse backup; OSPF costs of 10 and 100 express preferred and backup path behavior in the draft. The QoS policy is only a local example and must match carrier markings and bandwidth.

Global WAN

The Global profile combines four cities with a cloud boundary. It demonstrates multi-area OSPF internally and eBGP at the external boundary, with a /16 enterprise aggregate and a Null0 discard route. The VPN peer uses a documentation-only address and secret placeholders; replace them before any lab use. The BGP-to-OSPF redistribution is filtered to the declared remote prefix.

4. Decision logic: physical media

Distance is the first boundary. Reach labels and loss figures are planning inputs, not field acceptance results.

Run / service	Recommendation	Reason shown to the user	Review before deployment
0-100 m access	Cat6a balanced copper; 10GBASE-T / IEEE 802.3an.	Within the 100 m Ethernet channel limit; Cat6a supports 10GBASE-T when the full channel is compliant.	Patch leads, cable category, alien crosstalk, PoE class, bundle heat, and switch power budget.
101-400 m campus backbone	OM4 multimode; 10GBASE-SR at 850 nm.	OM4 provides modal bandwidth and nominal 400 m reach for this 10 GbE optical option.	Optic datasheet, connector count, polarity, loss, receive power, and dispersion.
Over 400 m to 10 km	OS2 single-mode; 10GBASE-LR candidate.	Single-mode plant avoids multimode reach limits at longer distances.	Actual module budget, splice/connector losses, dispersion, and route diversity.
Over 10 km to 40 km	OS2 single-mode; 10GBASE-ER candidate.	ER is a nominal metropolitan reach choice when the optical budget supports it.	Exact optic budget, receive level, dispersion, and any attenuation mitigation.
Beyond 40 km or managed WAN	Carrier-engineered optics or provider-defined EVPL/wavelength service.	The tool avoids claiming common LR/ER modules reach beyond their nominal class.	CIR, QoS mapping, latency, jitter, MTU, physical diversity, and service demarcation.

Hard boundary

When the input explicitly requests copper beyond 100 m, the engine raises PhysicalDistanceViolationError. The error explains that channel insertion loss and signal attenuation make the run unreliable and directs the user toward fiber or a carrier service.

Fiber estimate

```
estimated_loss_dB = distance_km * assumed_attenuation_dB_per_km
                  + connectors * 0.75 dB
                  + splices * 0.10 dB
                  + 3.0 dB engineering margin
```

The current planning defaults are 3.0 dB/km for OM4 and 0.35 dB/km for OS2. For example, the 35 km HQ-to-colo link estimates 19.05 dB using four connectors, eight splices, and 3 dB margin against a generic 23 dB assumed budget. This is a screening calculation only; actual transceiver specifications and measured path data take precedence.

> A recommendation is not a purchasing specification. Confirm fiber type, wavelength, transceiver SKU, optical power budget, and carrier service details with the real design inputs.

5. Decision logic: VLSM and routing

The calculations are deterministic, exposed in the reports, and covered at boundary conditions by unit tests.

IPv4 VLSM allocation

```
required_total = endpoint_count + 1 gateway + 2 reserved addresses
block_size = max(4, next_power_of_two(required_total))
prefix_length = 32 - log2(block_size)
```

The allocator ranks requests by required block size, places the largest aligned block first, and continues until all requests are assigned. It rejects exhaustion, verifies containment within the root prefix, and checks for overlap. The first usable address is the gateway; network and broadcast addresses are reserved. The plan reports mask, wildcard, usable range, capacity, address use, and utilization.

Boundary example: 63 endpoint hosts require 64 usable addresses after adding the gateway. Network and broadcast reservations make 66 total addresses necessary, so a 64-address /26 is too small; the next fitting block is /25.

Routing selection

Condition	Selection	Rationale
One site, one AS, fewer than 50 subnet requests	Single-area OSPF, Area 0.	Keeps link-state scope simple while allowing dynamic route recalculation.
Multiple sites	Multi-area OSPF, Area 0 backbone plus planned site areas.	Contains site routing scope; summary only at deliberate contiguous boundaries.
Cloud/provider AS boundary	Multi-area OSPF internally and eBGP externally.	Separates internal reachability from policy-controlled route exchange.
Single site with 50+ subnet requests	Multi-area OSPF design.	Assignment rule for scaling the topology; not a protocol maximum.

Summarization must be safe

The Global profile reserves 10.30.0.0/16 as an enterprise aggregate. Its draft installs a high-distance Null0 route and announces the aggregate with BGP. More-specific internal routes must exist and a discard route must not mask unplanned gaps. A summary can attract traffic for unused addresses, so the address plan and external routing policy need review.

ACL wildcard and packet flow

For a Cisco wildcard, zero bits must match and one bits are ignored. The project derives the wildcard as the inverse of the subnet mask. In the Campus example, 10.10.6.10 to management host 10.10.7.65 matches the explicit deny; 10.10.6.10 to 8.8.8.8 matches the following source-specific permit. The evaluator models this ACL decision only; it does not simulate route lookup, NAT, or a firewall.

6. Outputs and operating workflow

The command line is intentionally small, and output paths can be chosen for each run.

Artifact	Purpose	Who uses it
report.md	Human-readable rationale, media and routing choices, VLSM table, and review points.	Designer, reviewer, and project presenter.
address_plan.csv	Spreadsheet-friendly segment, mask, gateway, broadcast, wildcard, and utilization values.	Address-plan review and classroom discussion.
network_plan.json	Complete structured input and calculated plan.	Audit, repeatable generation, and future integrations.
core_switch.cfg	Layer 3 switch draft with VLANs, SVIs, trunking, rapid-PVST, OSPF, and declared ACLs.	Lab adaptation and command-by-command review.
edge_router.cfg	Routed handoffs, OSPF, Metro QoS example, and Global BGP/IPsec templates where applicable.	Routing and security review in a supported lab.

Common commands

```
# List profiles
python -m netarchitect list-profiles

# Generate a preset
python -m netarchitect generate campus --out output/campus

# Generate the editable custom example
python -m netarchitect generate-custom examples/custom-profile.json --out output/custom

# Walk a sample ACL decision
python -m netarchitect flow campus --acl GUEST-TO-MANAGEMENT \
  --source 10.10.6.10 --destination 10.10.7.65

# Run the complete software verification wrapper
powershell -NoProfile -ExecutionPolicy Bypass -File .\verify_project.ps1
```

The direct-file entry point also works from the repository root: python -u .\netarchitect__main__.py list-profiles. Without a command, it lists the available profiles.

Repository map

Location	Contents
netarchitect/	CLI, models, validators, engines, renderers, and algorithms.
netarchitect/profiles/	The Campus, Metro, and Global preset JSON inputs.
tests/	Unit suite for media, VLSM, routing, validation, ACLs, and exports.
docs/	Strategic plan, algorithms, architecture, user guide, assumptions, and AI audit.
examples/	Custom profile and pre-generated sample artifacts for each preset.
verify_project.ps1	One-run software verification and artifact check.

7. Feasibility and practical uses

NetArchitect is feasible as a course prototype because the scope is deterministic and the runtime footprint is small.

Dimension	Assessment	Practical implication
Technical	Python 3.10+ and standard-library runtime; JSON input; small local calculations.	Runs on a student laptop without a server or network access.
Economic	No licensed UI, database, or vendor SDK required.	Low prototype operating cost; real network hardware and services are separate project costs.
Operational	CLI commands, generated files, and explicit assumptions.	Simple to rerun, inspect, submit, and discuss in a lab.
Academic	Modular algorithms, rationale, tests, and AI audit.	Supports explanation and defense rather than an opaque output.
Production readiness	Not production-ready by itself.	Needs inventory, platform syntax checks, route/security policy, carrier data, and lab validation.

Where it is useful

Use case	Value
Classroom design exercise	Shows how distance, endpoint counts, routing scope, and policy affect a proposed design.
Early-stage campus or branch planning	Creates a first-pass VLAN/IP plan and identifies physical distance boundaries.
Address allocation review	Makes gateway reservation, block size, overlap, broadcast, and utilization visible.
Routing and ACL demonstration	Provides a compact OSPF/eBGP rationale and traceable ACL wildcard example.
Change review or lab preparation	Exports repeatable artifacts for peer review before a simulation or hardware lab.

What makes the proposal feasible

- The algorithmic boundary is clear: profiles are data, decisions are pure functions, and exports are rendered from a single plan.
- The standard-library runtime keeps installation and dependency failures low.
- The system does not need privileged access or connect to a network to produce an educational plan.
- The current scope matches the time-boxed assignment and leaves deeper simulation as a natural extension.

8. Verification evidence, limitations, and risk

The project has useful software checks. Passing them is not equivalent to a device-lab acceptance test.

Evidence	Observed result	What it supports
Python unit suite	26 tests passed; 0 failures in the user's consolidated run.	Media boundaries, VLSM capacity/exhaustion, routing rules, validation, ACL wildcard math, profile plans, and shared exports.
Preset generation	Campus, Metro, and Global exports generated successfully.	The end-to-end planning pipeline can create the expected report, CSV, JSON, and two IOS drafts per preset.
Custom generation	Custom Lab Network export generated successfully.	The custom JSON path parses and writes all standard artifacts.
ACL packet flow	Guest-to-management denied; guest-to-public-DNS permitted by this ACL.	The simplified ACL evaluator and generated wildcard decisions match expected examples.
Artifact check	All expected files and the AI audit log found.	Verification wrapper checks hand-in completeness at the file level.

AI audit highlights

- A naive VLSM formula used the endpoint count without reserving gateway, network, and broadcast addresses. For 63 endpoints it suggests /26; the corrected engine sizes to /25.

- A draft ACL used a subnet mask as an IOS wildcard. The corrected wildcard is the inverse mask, and packet-flow checks show how the error reaches implicit deny.

Key limitations

Limit	Why it matters	Next evidence needed
Generic optical loss	Attenuation and module budgets vary by optic, wavelength, path, and installation.	Selected module datasheet, full channel inventory, and measured optical power.
Logical device drafts	One core switch and one logical edge do not represent every site device or platform.	Actual topology, switch/router models, software versions, and per-site configs.
Carrier performance	Distance and local QoS cannot guarantee a sub-5 ms end-to-end path.	Written SLA plus latency, jitter, and failover measurements under load.
Security samples	ACL and IPsec examples are limited; VPN secrets and peer values are placeholders.	Security architecture, production key management, peer identity, and policy review.
No simulator integration	Passing code tests cannot prove packet forwarding on real IOS.	Packet Tracer/GNS3 validation or a controlled hardware lab.

Evidence statement for a presentation: 'The automated suite passed 26 tests, and the preset/custom export workflow plus ACL examples were exercised. Device parser and end-to-end packet behavior remain future lab validation.'

9. Presentation notes and likely questions

Use these points to explain the engineering choices in your own words. Be ready to show the calculations, not only the generated files.

A 60-second project explanation

NetArchitect is a Python CLI that turns three network scenarios or custom JSON into a documented design, a VLSM address plan, and Cisco IOS drafts. The engine explains its media, routing, and subnet decisions and validates important boundaries before generating output. The project separates validation, algorithms, and rendering so one plan feeds every artifact. The 26-test suite passed, including distance boundaries, address exhaustion, routing thresholds, and ACL wildcard checks. It is an educational planning aid; I would validate the generated configs in a device-specific lab before deployment.

Likely questions and concise answers

Question	Answer to defend
Why make it a CLI?	The assignment allows a CLI, dashboard, or API. A CLI keeps the runtime small, makes decisions repeatable, and keeps the calculation path visible.
How do you prevent subnet overlap?	Requests are ranked from largest to smallest, allocated on aligned power-of-two boundaries, constrained to the root prefix, then checked for overlap.
Why does a 63-host request need /25?	It needs 63 endpoint IPs plus one gateway and reserves network and broadcast, for 66 total addresses. A /26 has only 62 usable addresses.
Why use OM4 at 260 m?	It sits inside the modeled campus multimode range and pairs with 10GBASE-SR at 850 nm, subject to the optic's actual reach and link budget.
Why switch to OS2 for Metro?	The 15-35 km design distances exceed campus multimode reach; OS2 with a suitable LR/ER optic is the candidate, subject to measured optical budget.
How is the Metro backup selected?	The draft gives the primary path a lower OSPF cost and the diverse backup a higher cost, so the preferred route is chosen while the alternate remains available.
Why eBGP in Global?	The profile crosses a cloud/provider autonomous-system boundary. OSPF serves internal reachability and eBGP supports external policy exchange.
What do the 26 tests prove?	They verify modeled software rules and outputs. They do not prove device parser compatibility, carrier SLA, or end-to-end forwarding.
What would you improve next?	Generate per-site device configurations, validate against specific IOS/IOS XE versions in Packet Tracer/GNS3, add topology diagrams, and accept measured optical/module data.

Recommended demo order

- List the three profiles and show one preset's scenario parameters.
- Open its report and trace one medium decision, one VLSM block, and one routing explanation.
- Show the matching SVI and ACL lines in the switch draft, then walk the deny and permit packet examples.
- Show the 26 passing tests and explain the distinction between unit evidence and a simulator/device lab.

10. Final summary and hand-in checklist

The project is ready to explain as a tested software prototype, with deployment readiness explicitly left to a device-specific lab.

Final summary

NetArchitect provides a coherent, explainable workflow for the CIC-307 brief. It has three representative profiles, validates custom parameters, makes media and routing choices with rationale, performs aligned IPv4 VLSM with gateway and wildcard calculations, renders IOS switch/router drafts, and documents two corrected AI failure cases. The submitted verification run reports 26 passing tests, successful preset and custom exports, expected ACL decisions, and the required output files.

The implementation is feasible for the course because it is local, dependency-light, reproducible, and modular. Its most important engineering boundary is explicit: it narrows the design space and prepares a reviewable draft, but it does not replace inventory, vendor syntax review, a real optical budget, a carrier SLA, or a network simulator.

> One-sentence conclusion: NetArchitect converts network requirements into a transparent first-pass design and configuration package, with automated checks that support learning and review while leaving deployment validation to the engineer.

Hand-in documentation checklist

- README and strategic plan are present.
- Architecture, algorithm, user, and assumptions documents are present.
- Unit-test suite is included; the reported run passed 26 tests.
- AI audit log records the VLSM and wildcard-mask failure cases.
- Preset exports exist for Campus, Metro, and Global, with one custom-profile export.
- Generated IOS drafts are described as reviewable examples, not production-ready config.

Source materials

Primary project specification: NetArchitect Assignment Specification-2.pdf, supplied with the project. Supporting project documentation: README.md; docs/STRATEGIC_PLAN.md; docs/ARCHITECTURE.md; docs/ALGORITHMS.md; docs/USER_GUIDE.md; docs/ASSUMPTIONS_AND_LIMITS.md; docs/ai_audit_log.md.

END OF PROJECT HANDBOOK